

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Electrónica y Automatización Asignatura: Fundamentos de Programación NRC: 28561
	Apellidos y nombres del estudiante: Figueroa Sofía Santiago Velasco Tigse Alexander

Taller 4

Laboratorio propuesto (entregable práctico)

Antecedentes:

En este taller trabajamos con el manejo de estructuras y archivos de texto en lenguaje C. A partir de un registro de estudiantes, desarrollamos un programa CRUD que permite agregar, listar, consultar, actualizar y eliminar información. Además, los datos se guardan en un archivo estudiantes.txt dentro de una carpeta llamada Agenda.

Objetivo: implementar el CRUD completo y probarlo con al menos 10 registros.

1. Crear el proyecto en Code:Blocks y pegar el código en un solo archivo main.c.
2. Ejecutar y agregar 3 estudiantes. Verificar que se cree estudiantes.txt y que cada estudiante quede en una línea.
3. Probar la validación de ID duplicado (intentar agregar el mismo ID dos veces).
4. Listar y verificar el formato de tabla en consola.
5. Consultar por ID existente y por uno inexistente. Fundamentos de Programación
Página 9
6. Actualizar un estudiante y verificar que los cambios persistan al listar de nuevo.
7. Eliminar un estudiante y verificar que ya no aparece.

8. Abrir estudiantes.txt con un editor de texto y comprobar que el formato sigue siendo “;” por campos.

Código:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <direct.h>

#define ARCHIVO_EST "Agenda/estudiantes.txt"

typedef struct
{
    char id[20];
    char apellidos[50];
    char nombres[50];
    int edad;
} Estudiante;

/* Elimina el salto de linea que deja fgets */
void limpiarNuevaLinea(char cadena[])
{
    cadena[strcspn(cadena, "\n")] = '\0';
}

/* Lee una cadena de forma segura */
void leerCadena(const char mensaje[], char destino[], int tamano)
{
    printf("%s", mensaje);
    fgets(destino, tamano, stdin);
    limpiarNuevaLinea(destino);
}
```

```

/* Lee un numero entero */
int leerEntero(const char mensaje[])
{
    char cadena[50];

    printf("%s", mensaje);

    fgets(cadena, sizeof(cadena), stdin);

    return atoi(cadena);
}

/* Convierte una linea del archivo en un estudiante */
int parsearEstudiante(const char linea[], Estudiante *e)
{
    int campos;

    campos = sscanf(linea, "%19[^\n];%49[^\n];%49[^\n];%d",
                    e->id,
                    e->apellidos,
                    e->nombres,
                    &e->edad);

    return campos == 4;
}

/* Verifica si un ID ya existe */
int existeId(const char idBuscado[])
{
    FILE *archivo;

    char linea[200];

    Estudiante e;

    archivo = fopen(ARCHIVO_EST, "r");

```

```

    if (archivo == NULL)
    {
        return 0;
    }
    while (fgets(linea, sizeof(linea), archivo))
    {
        if (parsearEstudiante(linea, &e))
        {
            if (strcmp(e.id, idBuscado) == 0)
            {
                fclose(archivo);
                return 1;
            }
        }
    }
    fclose(archivo);
    return 0;
}

/* Agrega un estudiante */
void agregarEstudiante()
{
    FILE *archivo;

    Estudiante nuevo;

    printf("\n--- AGREGAR ESTUDIANTE ---\n");

    leerCadena("Ingrese ID: ", nuevo.id, sizeof(nuevo.id));

    if (existeld(nuevo.id))
    {

```

```

        printf("Error: ya existe un estudiante con ese ID.\n");

        return;
    }

    leerCadena("Ingrese apellidos: ",
               nuevo.apellidos,
               sizeof(nuevo.apellidos));

    leerCadena("Ingrese nombres: ",
               nuevo.nombres,
               sizeof(nuevo.nombres));

    nuevo.edad = leerEntero("Ingrese edad: ");

    archivo = fopen(ARCHIVO_EST, "a");

    if (archivo == NULL)
    {
        printf("Error: no se pudo abrir el archivo.\n");

        return;
    }

    fprintf(archivo, "%s;%s;%s;%d\n",
            nuevo.id,
            nuevo.apellidos,
            nuevo.nombres,
            nuevo.edad);

    fclose(archivo);

    printf("Estudiante agregado correctamente.\n");
}

/* Lista todos los estudiantes */

void listarEstudiantes()
{

```

```
FILE *archivo;

char linea[200];

Estudiante e;

int cantidad = 0;

archivo = fopen(ARCHIVO_EST, "r");

if (archivo == NULL)

{

    printf("\nNo existen estudiantes registrados.\n");

    return;

}

printf("\n%-15s %-20s %-20s %-10s\n",

        "ID", "APELLIDOS", "NOMBRES", "EDAD");

printf("-----\n");

while (fgets(linea, sizeof(linea), archivo))

{

    if (parsearEstudiante(linea, &e))

    {

        printf("%-15s %-20s %-20s %-10d\n",

                e.id,

                e.apellidos,

                e.nombres,

                e.edad);

        cantidad++;

    }

}
```

```

fclose(archivo);

if (cantidad == 0)
{
    printf("No existen estudiantes registrados.\n");
}
}

/* Busca un estudiante por ID */
int buscarPorId(const char idBuscado[], Estudiante *resultado)
{
    FILE *archivo;
    char linea[200];
    Estudiante e;
    archivo = fopen(ARCHIVO_EST, "r");
    if (archivo == NULL)
    {
        return 0;
    }
    while (fgets(linea, sizeof(linea), archivo))
    {
        if (parsearEstudiante(linea, &e))
        {
            if (strcmp(e.id, idBuscado) == 0)
            {
                *resultado = e;
                fclose(archivo);
                return 1;
            }
        }
    }
}

```

```

    }

}

fclose(archivo);

return 0;

}

/* Consulta un estudiante por ID */

void consultarEstudiante()

{

    char idBuscado[20];

    Estudiante e;

    printf("\n--- CONSULTAR ESTUDIANTE ---\n");

    leerCadena("Ingrese el ID que desea buscar: ",

               idBuscado,

               sizeof(idBuscado));

    if (buscarPorId(idBuscado, &e))

    {

        printf("\nEstudiante encontrado:\n");

        printf("ID: %s\n", e.id);

        printf("Apellidos: %s\n", e.apellidos);

        printf("Nombres: %s\n", e.nombres);

        printf("Edad: %d\n", e.edad);

    }

    else

    {

        printf("No se encontro un estudiante con ese ID.\n");

    }

}

```



```

/* Actualiza un estudiante */

void actualizarEstudiante()

{
    FILE *archivo;

    FILE *temporal;

    char linea[200];

    char idActualizar[20];

    Estudiante e;

    int actualizado = 0;

    printf("\n--- ACTUALIZAR ESTUDIANTE ---\n");


    leerCadena("Ingrese el ID que desea actualizar: ",
               idActualizar,
               sizeof(idActualizar));

    archivo = fopen(ARCHIVO_EST, "r");

    if (archivo == NULL)
    {
        printf("No existe el archivo de estudiantes.\n");

        return;
    }

    temporal = fopen("Agenda/tmp.txt", "w");

    if (temporal == NULL)
    {
        fclose(archivo);

        printf("No se pudo crear el archivo temporal.\n");

        return;
    }
}

```

```
while (fgets(linea, sizeof(linea), archivo))
{
    if (parsearEstudiante(linea, &e))
    {
        if (strcmp(e.id, idActualizar) == 0)
        {
            printf("\nIngrese los nuevos datos:\n");

            leerCadena("Nuevos apellidos: ",
                      e.apellidos,
                      sizeof(e.apellidos));

            leerCadena("Nuevos nombres: ",
                      e.nombres,
                      sizeof(e.nombres));

            e.edad = leerEntero("Nueva edad: ");

            actualizado = 1;
        }

        fprintf(temporal, "%s;%s;%s;%d\n",
                e.id,
                e.apellidos,
                e.nombres,
                e.edad);
    }
}

fclose(archivo);

fclose(temporal);

remove(ARCHIVO_EST);

rename("Agenda/tmp.txt", ARCHIVO_EST);
```

```

    if (actualizado)
    {
        printf("Estudiante actualizado correctamente.\n");
    }

    else
    {
        printf("No se encontro un estudiante con ese ID.\n");
    }
}

/* Elimina un estudiante */
void eliminarEstudiante()
{
    FILE *archivo;
    FILE *temporal;
    char linea[200];
    char idEliminar[20];
    Estudiante e;
    int eliminado = 0;
    printf("\n--- ELIMINAR ESTUDIANTE ---\n");
    leerCadena("Ingrese el ID que desea eliminar: ",
        idEliminar,
        sizeof(idEliminar));
    archivo = fopen(ARCHIVO_EST, "r");
    if (archivo == NULL)
    {
        printf("No existe el archivo de estudiantes.\n");
        return;
    }

```

```

}

temporal = fopen("Agenda/tmp.txt", "w");

if (temporal == NULL)

{

    fclose(archivo);

    printf("No se pudo crear el archivo temporal.\n");

    return;

}

while (fgets(linea, sizeof(linea), archivo))

{

    if (parsearEstudiante(linea, &e))

    {

        if (strcmp(e.id, idEliminar) != 0)

        {

            fprintf(temporal, "%s;%s;%s;%d\n",

                e.id,

                e.apellidos,

                e.nombres,

                e.edad);

        }

        else

        {

            eliminado = 1;

        }

    }

}

fclose(archivo);

```

```

fclose(temporal);

remove(ARCHIVO_EST);

rename("Agenda/tmp.txt", ARCHIVO_EST);

if (eliminado)
{
    printf("Estudiante eliminado correctamente.\n");
}

else
{
    printf("No se encontro un estudiante con ese ID.\n");
}
}

```

/* Muestra el menu */

```

int mostrarMenu()
{
    int opcion;

    printf("\n===== \n");

    printf(" CRUD DE ESTUDIANTES\n");

    printf("===== \n");

    printf("1. Agregar estudiante\n");

    printf("2. Listar estudiantes\n");

    printf("3. Consultar estudiante por ID\n");

    printf("4. Actualizar estudiante\n");

    printf("5. Eliminar estudiante\n");

    printf("0. Salir\n");

    printf("===== \n");
}

```

```
opcion = leerEntero("Seleccione una opcion: ");  
  
return opcion;  
  
}  
  
int main()  
{  
  
    int opcion;  
  
    /* Crea la carpeta Agenda si no existe */  
    _mkdir("Agenda");  
  
    do  
    {  
  
        opcion = mostrarMenu();  
  
        switch (opcion)  
        {  
  
            case 1:  
  
                agregarEstudiante();  
  
                break;  
  
            case 2:  
  
                listarEstudiantes();  
  
                break;  
  
            case 3:  
  
                consultarEstudiante();  
  
                break;  
  
            case 4:  
  
                actualizarEstudiante();  
  
                break;  
  
            case 5:  
  
                eliminarEstudiante();
```

```

        break;

    case 0:

        printf("\nSaliendo del programa...\n");

        break;

    default:

        printf("\nOpcion no valida.\n");

        break;

    }

} while (opcion != 0);

return 0;

}

```

Validación del Código:

```

C:\Users\user\Desktop\Untitled1.exe
=====
CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion: 2

ID      APELLIDOS      NOMBRES      EDAD
-----
1000457027  Figueroa      Sofia      19
100456789  Velasco      Eduardo      20
100456773  Atencia      Dafne      20
100456834  Tigse      Dicson      19
100456826  Santiago      Velasco      20
10004531  Yunga      Andriu      20

=====
CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion:

```

```
C:\Users\user\Desktop\Untitled1.exe
=====
Seleccione una opcion: 2

ID      APELLIDOS      NOMBRES      EDAD
-----
1000457027  Figueroa      Sofia        19
100456780  Velasco      Eduardo      20
100456773  Atencia      Dafne        20
100456834  Tigse        Dicson       19
100456826  Santiago     Velasco      20
10004531   Yunga        Andriu       20
=====

CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion: 3

--- CONSULTAR ESTUDIANTE ---
Ingrese el ID que desea buscar: 100456773

Estudiante encontrado:
ID: 100456773
Apellidos: Atencia
Nombres: Dafne
Edad: 20

=====

CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion:
```

```
C:\Users\user\Desktop\Untitled1.exe
=====
Seleccione una opcion: 4

--- ACTUALIZAR ESTUDIANTE ---
Ingrese el ID que desea actualizar: 100456826

Ingrese los nuevos datos:
Nuevos apellidos: Fernandez
Nuevos nombres: Pedrito
Nueva edad: 20
Estudiante actualizado correctamente.

=====

CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion: 2

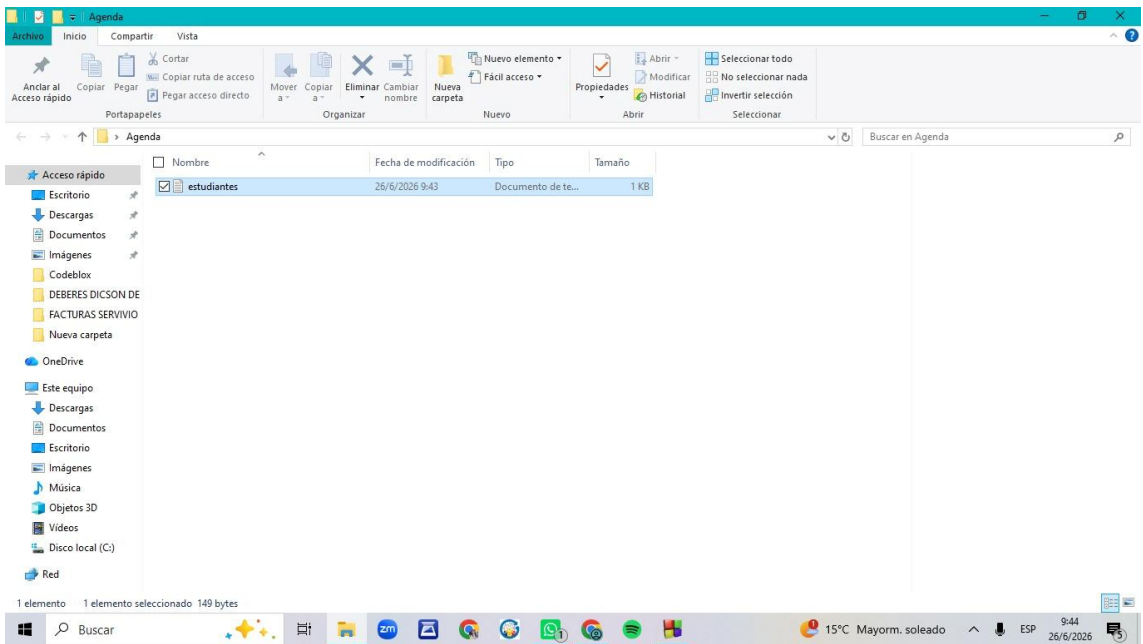
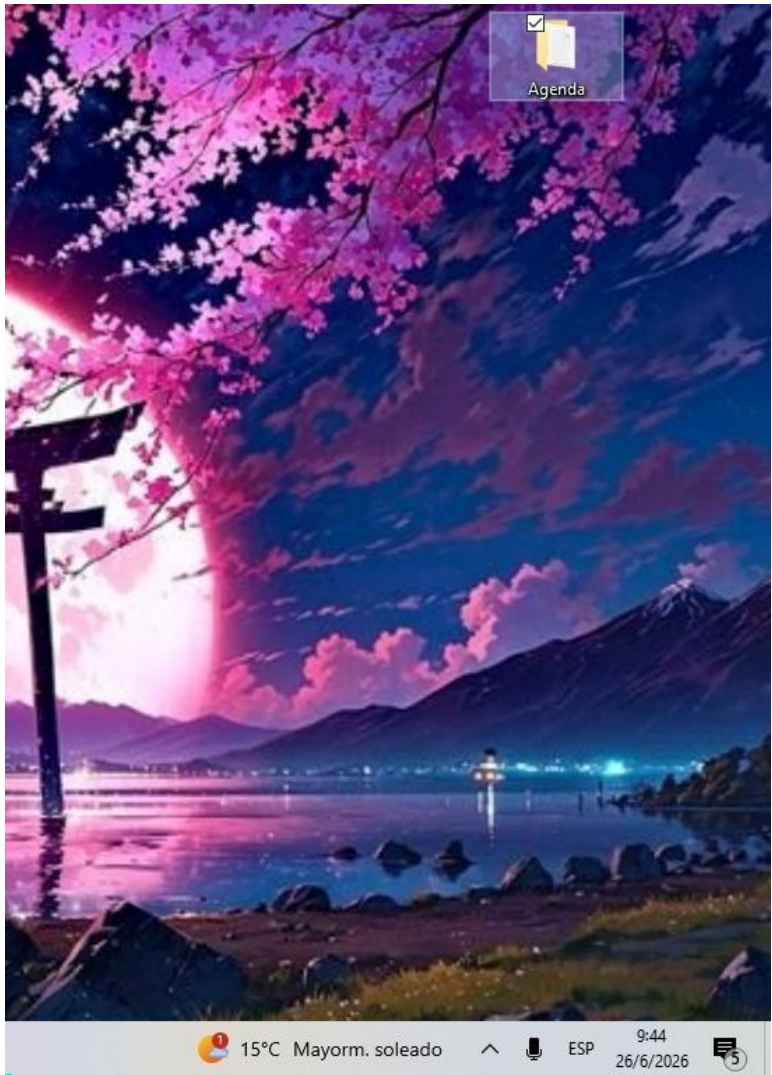
ID      APELLIDOS      NOMBRES      EDAD
-----
1000457027  Figueroa      Sofia        19
100456780  Velasco      Eduardo      20
100456773  Atencia      Dafne        20
100456834  Tigse        Dicson       19
100456826  Fernandez    Pedrito      20
10004531   Yunga        Andriu       20
=====

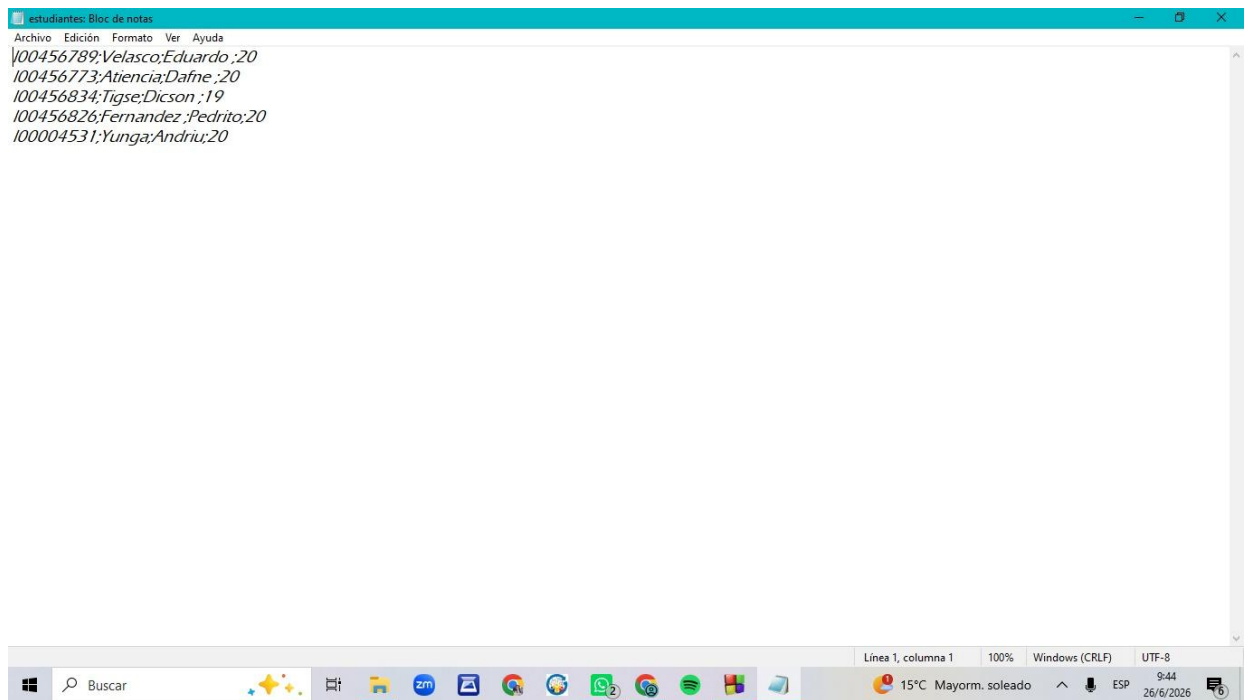
CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion:
```



```
C:\Users\user\Desktop\Untitled1.exe
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion: 5
--- ELIMINAR ESTUDIANTE ---
Ingrese el ID que desea eliminar: 1000457027
Estudiante eliminado correctamente.
=====
CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion: 2
ID      APELLIDOS      NOMBRES      EDAD
-----
100456780 Velasco      Eduardo      20
100456773 Atencia      Dafne        20
100456834 Tigse        Dicson       19
100456826 Fernandez    Pedrito      20
100004531 Yunga        Andriu       20
=====
CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion:
```

```
C:\Users\user\Desktop\Untitled1.exe
=====
Seleccione una opcion: 5
--- ELIMINAR ESTUDIANTE ---
Ingrese el ID que desea eliminar: 1000457027
Estudiante eliminado correctamente.
=====
CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion: 2
ID      APELLIDOS      NOMBRES      EDAD
-----
100456780 Velasco      Eduardo      20
100456773 Atencia      Dafne        20
100456834 Tigse        Dicson       19
100456826 Fernandez    Pedrito      20
100004531 Yunga        Andriu       20
=====
CRUD DE ESTUDIANTES
=====
1. Agregar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opcion: 0
Saliendo del programa...
Process returned 0 (0x0)   execution time : 926.800 s
Press any key to continue.
```





```
estudiantes: Bloc de notas
Archivo Edición Formato Ver Ayuda
100456789;Velasco;Eduardo;20
100456773;Atencia;Dafne;20
100456834;Tigse;Dicson;19
100456826;Fernandez;Pedrito;20
100004531;Yunga;Andriu;20
```

Linea 1, columna 1 100% Windows (CRLF) UTF-8
15°C Mayorm. soleado 9:44 26/6/2026

Conclusión:

Con este taller logramos comprender cómo implementar un CRUD en lenguaje C para manejar registros de estudiantes. El programa permite agregar, listar, consultar, actualizar y eliminar información, además de guardar los datos en un archivo de texto para que no se pierdan al cerrar el programa.

Recomedación:

Se recomienda probar cada opción del programa con varios registros y verificar que los datos se guarden correctamente en el archivo `estudiantes.txt`. También es importante revisar que no se repitan los ID y comprobar el funcionamiento de las opciones de actualización y eliminación antes de entregar el taller.